JOHANNES KEPLER
UNIVERSITY LINZ

Richard Degenfellner, BSc
K11949118
Institute of Machine
Learning

@ K11949118@students.jku.at

September 19, 2026

TiRex methods analyzed with different loss functions.



Technical Report

Institute of Machine Learning

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

Abstract **3**

1. Introduction **4**

2. Methods and Experiments **4**
2.1 Training and Test data 5
2.2 Loss functions 5
2.3 Autoregressive 6
2.4 Custom TiRex model 6
2.4.1 CPM and MH 6
2.4.2 xLSTM vs. LSTM 7

3. Results **7**
3.1 Metrics 7
3.1.1 sMAPE 7
3.1.2 Quantile Loss 10
3.2 Autoregressive 10
3.3 CPM 11
3.4 xLSTM vs. LSTM 11
3.5 Best vs. Last Test Loss 11

4. Conclusion **12**

References **13**

Appendix **14**
A.1 Train and Validation Losses, Fine-tuning pre-trained model on single sequence 14
A.2 Train and Validation Losses, Custom TiRex on big dataset 14
A.3 Train and Validation Losses, Custom TiRex on single sequence 14

Abstract

Time series foundation models promise strong zero-shot forecasting performance. This work investigates the TiRex foundation model. We compare three loss functions (quantile, MSE, L1), the model's original multi-patch forecasting behavior against an autoregressive generation strategy, and two dataloader masking strategies (Contiguous Patch Masking vs. Mask Horizon). We also train a strongly parameter-reduced custom TiRex variant (1.6 million vs. 35 million parameters) from scratch and compare an xLSTM layer against a standard LSTM layer within the same architecture. All experiments are evaluated on a combined dataset of household, national, and European energy time series. Results show that L1 loss consistently yields the best SMAPE scores while quantile loss performs worst, that autoregressive generation improves performance for long forecast horizons but induces quantile collapse, that CPM clearly outperforms the mask horizon strategy, and that the LSTM layer performs slightly better than sLSTM under identical hyperparameters. While the custom, parameter-reduced model cannot match the pre-trained TiRex model's performance, it still learns meaningful forecasting behavior. However, fine-tuning the pre-trained model on a specific task or sequence yields a noticeably larger performance gain, suggesting that fine-tuning the existing foundation model is the more effective route compared to training a smaller custom model from scratch. The work can be found in the GitHub repository: https://github.com/Richi0D/tirex_loss

1. Introduction

Accurate time series forecasting is a core requirement in many domains, such as load forecasting, renewable generation forecasting and price forecasting. Good forecasting methods support grid stability, resource planning and market operations. Traditionally, such forecasting problems have been addressed with statistical models or deep learning models trained from scratch for a specific dataset or task.

Recently, time series foundation models, large models pre-trained on diverse collections of time series, have emerged as a promising alternative, offering zero-shot forecasting capability without task-specific training. TiRex (Auer et al. [3]), a foundation model built on the xLSTM architecture (Beck et al. [4]), is one such recent model and achieves strong zero-shot performance on general forecasting benchmarks.

This work (https://github.com/Richi0D/tirex_loss) investigates the TiRex model in the context of energy time series forecasting through several practical modifications and experiments: (i) fine-tuning the pre-trained model on energy data with different loss functions, (ii) comparing the model's original multi-patch forecasting behavior with an autoregressive generation strategy, (iii) training a strongly parameter-reduced custom variant of TiRex from scratch, (iv) comparing two dataloader masking strategies for training, and (v) comparing the xLSTM layer with a standard LSTM layer within the same architecture. All experiments are evaluated on a combined dataset of energy time series from household, national, and European sources. The remainder of this paper is structured as follows: section 2 describes the data, methods, and experimental setup; section 3 presents and discusses the results; and section 4 summarizes the findings and outlines directions for future work.

2. Methods and Experiments

This section describes all the methods, experiments and modifications to the TiRex model by Auer et al. [3].

For all fine-tuning and training tasks, an AdamW optimizer (Loshchilov and Hutter [7]) is used. See Table 1 for hyperparameter settings.

Table 1: Hyperparameter configurations for TiRex model variants.

Hyperparameter	**Fine-tuned (Pre-trained)**	**Custom Model**
Learning Rate	1×10^{-4}	1×10^{-4}
Weight Decay	1×10^{-2}	1×10^{-2}
Gradient Clipping	None	Max Norm: 1.0
LR Scheduler	ExponentialLR ($\gamma = 0.99$)	ExponentialLR ($\gamma = 0.99$)

The pre-trained TiRex model is fine-tuned for a maximum of 15 epochs. The custom model is trained for a maximum of 100 epochs. All main evaluations use the model checkpoint with the best test loss during training, which means some models were effectively trained for fewer epochs than others. A separate comparison between the best and the last test loss is given in section 3.5. Detailed loss scores and plots are provided in Appendix A.

2.1 Training and Test data

All datasets used in this work are from the energy domain. They contain time series at various intervals ranging from 5 to 60 minutes.

- Electricity consumption of a single two-person household in Austria, collected from the household meter. The time series has a 15-minute interval and contains around 100k timestamps.
- OPSD Time Series (Europe) [9]: Energy load, energy generation, and price time series from 32 European countries. Only the stacked 15- and 60-minute datasets are used.
- Open Energy Hub [11]: A renewable energy and electricity demand dataset with exogenous variables at 5-minute intervals. Only the production and demand columns are used.
- FETS Dataset [8]: Contains time series from the European and international energy domain.

These individual sources were combined into a single dataset containing 361 different series in total. For each series, the last 30% of timestamps are held out for testing and evaluation and further split with a ratio of 50%. While the remaining 70% are used for training. This creates a 70%, 15%, 15% dataset split. In total, this amounts to around 17.7 million timesteps for training and around 7.6 million timesteps for testing and evaluation. However, due to computational limitations, only a subset of the full dataset is used in practice: to train and evaluate the custom TiRex model, only 72 (20%) of the 361 series are used. For fine-tuning the pre-trained TiRex foundation model, only a single time series, "DE Power - Load DE" from the FETS Dataset [8], is used, which contains 5 years of data at a 15-minute interval (around 200k timesteps).

A fixed random seed is used across all experiments to make the data splits deterministic and reproducible, which allows the different models to be compared on the exact same test dataset.

2.2 Loss functions

Three loss functions are used to explore their effect in the TiRex setting.

- Quantile Loss: with Q the quantile levels, y_t the true value at time t, $\hat{y}_t^q$ the quantile prediction for quantile level q, and h the forecast horizon.

$$L = \frac{1}{|Q| \cdot h} \sum_{t=1}^{h} \sum_{q \in Q} \begin{cases} q(y_t - \hat{y}_t^q) & \text{if } \hat{y}_t^q \leq y_t \\ (1-q)(\hat{y}_t^q - y_t) & \text{otherwise.} \end{cases} \tag{1}$$

- Mean squared error (MSE): with $q = 0.5$ (median) as the fixed quantile.

$$MSE = \frac{1}{h} \sum_{t=1}^{h} (y_t - \hat{y}_t^q)^2 \tag{2}$$

- L1 Loss (MAE): with $q = 0.5$ (median) as the fixed quantile.

$$L1 = \frac{1}{h} \sum_{t=1}^{h} |y_t - \hat{y}_t^q| \tag{3}$$

2.3 Autoregressive

A first look was given the generation method used for predictions, referred to as multi-patch prediction. The TiRex foundation model by Auer et al. [3] has a context length c of 2048 and predicts the next patch with output patch length of 32. If the forecast horizon h exceeds the output patch length, multiple patches must be predicted. For a given time series with length n and a forecast horizon $h > 32$ the TiRex model uses the last c timesteps of n, shifts the context window c by the output patch length and appends the previously predicted patch as missing (NaN) values. So, the longer the forecast horizon h gets, the less real context the model has available. If $h > c$ the context window is eventually filled entirely with zeros (NaN replacement value) and the model predicts only zeros.

Other existing pre-trained models (Das et al. [5]; Ansari et al. [1]) address this issue via autoregressive generation, using point estimates such as the mean or median. Therefore, the forecast generation method of the TiRex model is modified to use autoregressive generation: instead of inserting missing values, the median of the previously predicted patch is concatenated to the context c.

The pre-trained TiRex model was fine-tuned on a single time series, "DE Power - Load DE" from the FETS Dataset, using both the original behavior and the autoregressive method, across all three loss functions described in section 2.2. Fine-tuning was performed by sampling sequences with random horizon lengths between 32 and 128 and passing them through the prediction function, in which a multi-patch prediction is generated and the loss is computed against the true values.

2.4 Custom TiRex model

Due to limited computational resources, the TiRex model was reduced from around 35 million parameters to 1.6 million parameters via a custom implementation. It uses the same architecture, but with hidden sizes reduced by half, a single sLSTM block instead of 12, and a context length reduced to 1024, exactly half that of the pre-trained TiRex model.

The custom TiRex model was trained in various variants, using all loss functions from section 2.2. See section 2.4.1 and section 2.4.2 for details on the individual variants.

Two dataset variants were used to train the custom model: a larger dataset with 72 time series, and the single series "DE Power - Load DE" described in section 2.1, the latter enabling a direct comparison with the fine-tuned TiRex model. For training on the single series, the mask horizon strategy described in section 2.4.1 is used.

2.4.1 CPM and MH

Two different dataloader strategies are implemented for training. The first is Contiguous Patch Masking (CPM), as described in the TiRex paper by Auer et al. [3]. The second is mask horizon (MH), which replicates the multi-patch prediction behavior of the TiRex model. Mask horizon takes a sequence of length n plus a target length s and creates an input sequence by shifting the context by s and padding the shifted region with missing values. The last patch is used as the target to compute the loss. This is repeated for all multiples of the patch size 32 up to s_{max} ($s_{\text{shift}} = [32, 64, 96, 128, \ldots, 512]$). Mask horizon is illustrated in Figure 1. The maximum shift is defined as $s_{\max} = 512$ (half of the context length).

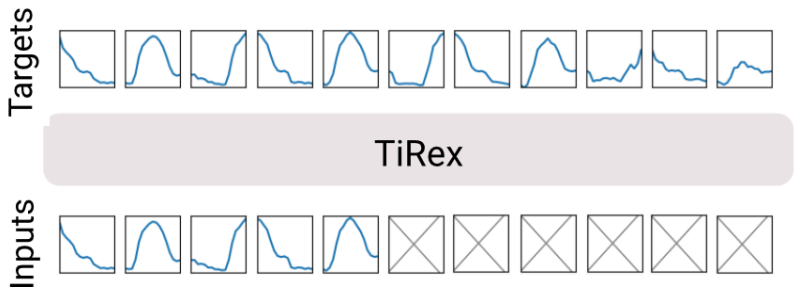


Figure 1: Mask horizon. Illustration adapted from Auer et al. [3]

2.4.2 xLSTM vs. LSTM

An additional point of interest is the effect of replacing the xLSTM layer (Beck et al. [4]) with a LSTM layer (Hochreiter and Schmidhuber [6]), while keeping the rest of the architecture unchanged. xLSTM introduced two new variants, the mLSTM and sLSTM cell (Beck et al. [4]). For TiRex and the custom implementation only the sLSTM cell is used. The sLSTM cell replaces the activations at the input and forget gate with exponential functions and introduces additional stabilization and normalization mechanisms to handle the large values from exponential activations.

3. Results

This section presents the results of the experiments described in section 2. Table 2 shows the scores for the models trained on the dataset of 72 series, and Table 3 shows the scores for the custom TiRex model and the fine-tuned TiRex model trained on a single sequence. Comparing SMAPE scores within each individual group, the L1 loss performs better in the custom TiRex model, while the quantile loss performs worst and the MSE loss slightly behind the L1 loss. One possible explanation is that the quantile loss requires the model to learn across all quantiles simultaneously, rather than concentrating on a single target value. Time series forecasting might benefit from simpler loss types. This effect is not noticeable in the fine-tuned version of TiRex. In this setting all losses achieve a similar score but improving the results compared to the original pre-trained version.

3.1 Metrics

Two main metrics are used to compare the different methods and experiments: the symmetric mean absolute percentage error (sMAPE) and the quantile loss. The sMAPE uses only the 50% (median) quantile of the model output to score against the target values, while the quantile loss uses all quantile predictions. For methods that do not produce quantile predictions, only the sMAPE is used.

3.1.1 sMAPE

As a general score the symmetric mean absolute percentage error (sMAPE) is used. The original definition from Armstrong [2] is slightly adapted and is defined as:

$$\text{SMAPE} = \frac{2}{n} \sum_{t=1}^{n} \frac{|F_t - A_t|}{|A_t| + |F_t|} \tag{4}$$

Table 2: Evaluation scores (big dataset)

			SMAPE Score		Quantile Score	
Dataloader	**Layer**	**Loss**	**Autoregressive**	**Original**	**Autoregressive**	**Original**
Custom TiRex model						
cpm	lstm	l1	0.4591 ± 0.0168	0.4819 ± 0.0052	-	-
cpm	lstm	mse	0.5289 ± 0.0067	0.5675 ± 0.0074	-	-
cpm	lstm	quantile	0.7024 ± 0.0066	0.7132 ± 0.0130	894.47 ± 7.77	897.51 ± 11.32
cpm	slstm	l1	0.4430 ± 0.0092	0.4529 ± 0.0087	-	-
cpm	slstm	mse	0.5337 ± 0.0063	0.5535 ± 0.0077	-	-
cpm	slstm	quantile	0.7032 ± 0.0072	0.7115 ± 0.0061	888.68 ± 6.16	913.33 ± 9.44
MH	lstm	l1	0.4638 ± 0.0108	0.4619 ± 0.0043	-	-
MH	lstm	mse	0.5646 ± 0.0045	0.5579 ± 0.0063	-	-
MH	lstm	quantile	0.7068 ± 0.0063	0.7170 ± 0.0037	897.45 ± 12.20	910.49 ± 10.93
MH	slstm	l1	0.4756 ± 0.0273	0.4497 ± 0.0089	-	-
MH	slstm	mse	0.5975 ± 0.0230	0.5846 ± 0.0258	-	-
MH	slstm	quantile	0.7176 ± 0.0152	0.7276 ± 0.0092	906.54 ± 9.95	938.39 ± 13.30
Original pre-trained TiRex model						
TiRex	slstm	quantile	0.4088	0.4042	445.70	354.85

[a] Mean ± std across 5 independent runs.

[b] Lower scores indicate better predictive performance across both metrics. Best score overall is marked in green and best score for each variant is marked in yellow.

Table 3: Evaluation scores (single sequence)

			SMAPE Score		Quantile Score	
Dataloader	**Layer**	**Loss**	**Autoregressive**	**Original**	**Autoregressive**	**Original**
Custom TiRex model						
MH	lstm	l1	0.0245 ± 0.0003	0.0231 ± 0.0004	-	-
MH	lstm	mse	0.0236 ± 0.0003	0.0223 ± 0.0003	-	-
MH	lstm	quantile	0.1347 ± 0.0006	0.1344 ± 0.0004	3556 ± 16.74	3545 ± 11.07
MH	slstm	l1	0.0277 ± 0.0006	0.0268 ± 0.0004	-	-
MH	slstm	mse	0.0281 ± 0.0004	0.0267 ± 0.0003	-	-
MH	slstm	quantile	0.1356 ± 0.0014	0.1355 ± 0.0006	3579 ± 40.79	3579 ± 15.82
Fine-tuned pre-trained TiRex model						
TiRex Or.	slstm	l1	0.0217 ± 0.0002	0.0218 ± 0.0002	-	-
TiRex Or.	slstm	mse	0.0224 ± 0.0002	0.0224 ± 0.0002	-	-
TiRex Or.	slstm	quantile	0.0218 ± 0.0005	0.0218 ± 0.0005	485.70 ± 15.40	467.15 ± 14.84
TiRex Auto.	slstm	l1	0.0218 ± 0.0003	0.0249 ± 0.0009	-	-
TiRex Auto.	slstm	mse	0.0219 ± 0.0003	0.0243 ± 0.0004	-	-
TiRex Auto.	slstm	quantile	0.0221 ± 0.0003	0.0234 ± 0.0003	472.57 ± 14.14	501.26 ± 11.58
Original pre-trained TiRex model						
TiRex	slstm	quantile	0.0350	0.0338	770.29	710.96

[a] Mean ± std across 5 independent runs.

[b] Lower scores indicate better predictive performance across both metrics. Best score overall is marked in green and best score for each variant is marked in yellow.

where t is the timestep, A_t the actual values and F_t the forecasted values, which in our case is either the model prediction for a single output value or the median of the quantile predictions. The advantage of the symmetric version is that it avoids exploding values when the actuals are close to zero and it treats over- and under-forecasts in a relative, symmetric way.

3.1.2 Quantile Loss

In addition to sMAPE, the quantile (pinball) loss is used to measure the quality of the individual quantile predictions. For a given quantile level $q \in Q$ and time step t, the quantile loss is computed using the same piecewise formulation as in Equation 1. The overall quantile loss reported for evaluation is obtained by averaging this loss over the forecast horizon $t = 1, \ldots, h$ and all evaluated quantile levels $q \in Q$.

3.2 Autoregressive

As shown in Table 2, the autoregressive method scores slightly better in most experiments. Only the original pre-trained TiRex model scores better with its original NaN-filling behavior (inserting missing values into the shifted context instead of the median prediction) for multi-patch prediction. But when trained on a single series this effect shifted in favor of the original NaN-filling behavior. Overall, Table 3 shows better performance for the custom TiRex model. It is also noticeable that when the pre-trained model is trained in an autoregressive way, only the autoregressive predictions can keep up with the original prediction method.

As shown in Figure 2, especially for a very long horizon that exceeds the model's context length, the autoregressive behavior still produces values that fit the ground truth well. The original behavior, in contrast, suffers from the problem that for long horizons, known past values are gradually shifted out of the model's context until only null values remain. The autoregressive behavior, on the other hand, suffers from quantile collapse: as shown in Figure 3, after the second patch (timestep > 32), once predictions from the previous step become part of the context, the quantiles get noticeably worse compared to the original behavior. And the effect becomes noticeably worse as the forecast horizon increases.

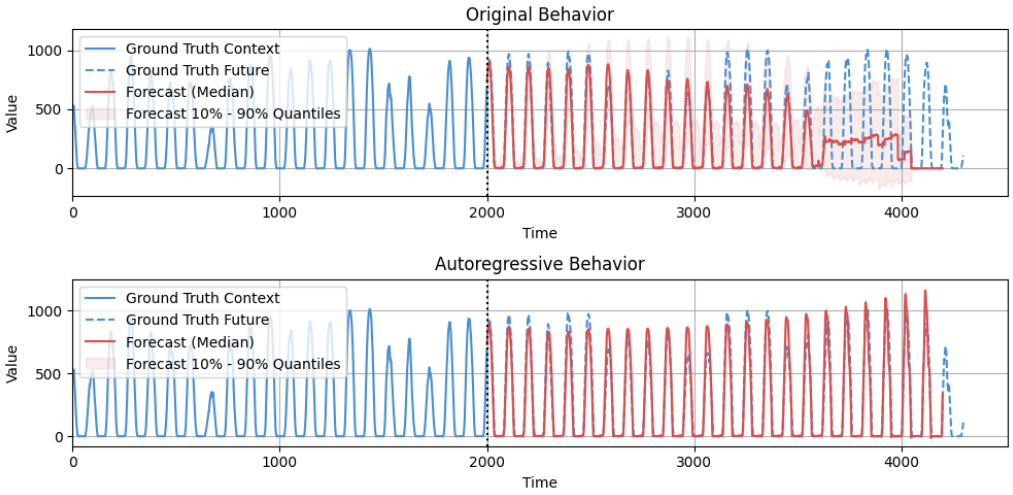


Figure 2: Comparison of autoregressive behavior vs. nan-filling method.

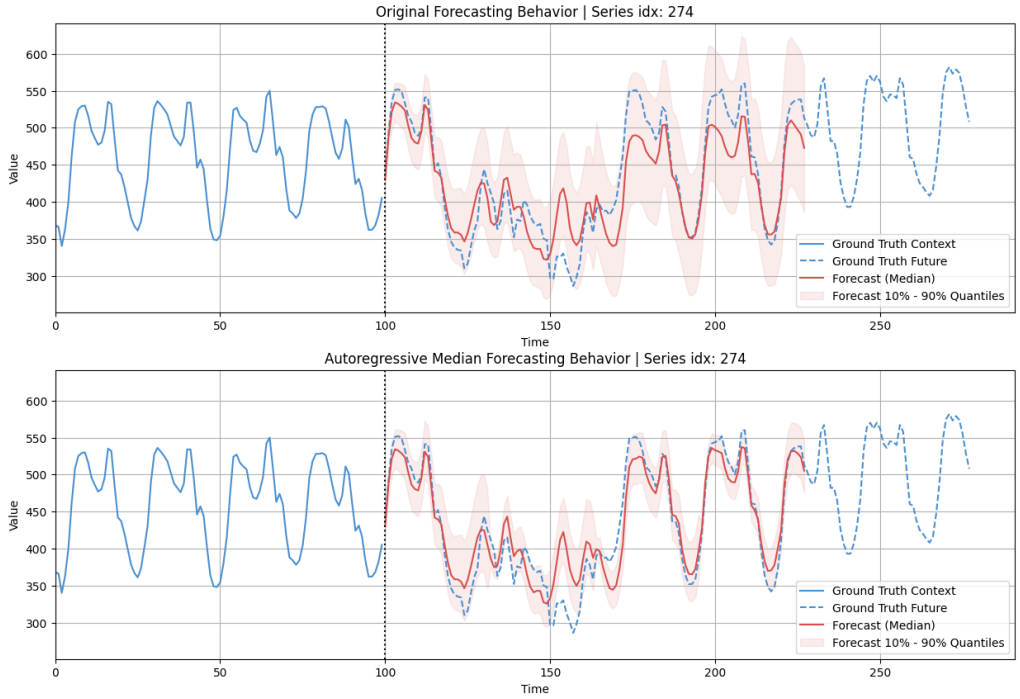


Figure 3: Comparison of autoregressive behavior vs. nan-filling method. On horizon $h = 192$.

3.3 CPM

CPM has a large impact on training and clearly improves model performance: all CPM scores in Table 2 are better than those of mask horizon. The models evidently learn better when trained with randomly scattered masked patches (CPM) than with a single, large masked block near the end of the sequence (MH). It appears that MH removes too much information from the context at once, making the task considerably harder to learn from. This was also noticeable during training that CPM improves the training loss and the deviation across multiple runs is much smaller, making it easier to find the best model.

3.4 xLSTM vs. LSTM

Interestingly, in both Table 2 and Table 3, the LSTM layer performs slightly better than the sLSTM layer in most variants. This could also be a training-related artifact, since the same hyperparameters were used for both variants and may not be equally well suited to each; alternatively, the LSTM layer may simply be easier to train than the sLSTM layer in this setting.

3.5 Best vs. Last Test Loss

In general, only the model checkpoint with the best evaluation loss is used for scoring the model. However, since all models are trained for the full 100 epochs, scores for the fully trained (last-epoch) models are also available and are shown in Table 4. These results show that it is generally better to use the checkpoint with the best test loss. Interestingly, they also show that quantile predictions tend to improve the longer the model is trained. A somewhat counterintuitive result that would benefit from further investigation.

Table 4: Evaluation scores (big dataset), Best vs. Last Test Loss

			SMAPE Score		Quantile Score	
Dataloader	**Layer**	**Loss**	**Best**	**Last**	**Best**	**Last**
Custom TiRex model						
cpm	lstm	l1	0.4819 ± 0.0052	0.4952 ± 0.0038	-	-
cpm	lstm	mse	0.5675 ± 0.0074	0.5908 ± 0.0047	-	-
cpm	lstm	quantile	0.7132 ± 0.0130	0.6949 ± 0.0024	897.51 ± 11.32	872.84 ± 4.31
cpm	slstm	l1	0.4529 ± 0.0087	0.4672 ± 0.0171	-	-
cpm	slstm	mse	0.5535 ± 0.0077	0.5654 ± 0.0183	-	-
cpm	slstm	quantile	0.7115 ± 0.0061	0.7173 ± 0.0150	913.33 ± 9.44	907.46 ± 10.00
MH	lstm	l1	0.4619 ± 0.0043	0.4794 ± 0.0036	-	-
MH	lstm	mse	0.5579 ± 0.0063	0.5609 ± 0.0055	-	-
MH	lstm	quantile	0.7170 ± 0.0037	0.7105 ± 0.0045	910.49 ± 10.93	901.63 ± 6.25
MH	slstm	l1	0.4497 ± 0.0089	0.4777 ± 0.0152	-	-
MH	slstm	mse	0.5846 ± 0.0258	0.6030 ± 0.0289	-	-
MH	slstm	quantile	0.7276 ± 0.0092	0.7281 ± 0.0240	938.39 ± 13.30	941.42 ± 19.79

[a] Mean ± std across 5 independent runs.
[b] Lower scores indicate better predictive performance across both metrics.
[c] Only scores from original multi-patch prediction behavior are shown.

4. Conclusion

This work explored several modifications to the TiRex time series foundation model, covering the choice of loss function, the forecast generation strategy, a strongly parameter-reduced custom model, the dataloader masking strategy, and the choice of recurrent layer.

Across all experiments for the custom TiRex model, L1 loss consistently produced the best SMAPE scores, while quantile loss consistently performed worst, likely because it requires the model to fit an entire predictive distribution rather than a single target value. Switching from the original NaN-filling multi-patch generation to autoregressive generation improved performance in most settings, specially using CPM as training method, and was particularly beneficial for forecast horizons exceeding the model's context length, where the original method degrades as known values are pushed out of context. This benefit came at the cost of quantile collapse over successive autoregressive steps. A combination of both methods could benefit the predictions, specially if the horizon is long. For shorter horizons, the standard multi-patch generation could be used, and after a certain forecast horizon is reached, the model could switch to autoregressive generation.

For the training data strategy, Contiguous Patch Masking (CPM) clearly outperformed the simpler Mask Horizon (MH), suggesting that exposing the model to many small, randomly distributed masked regions during training is more effective than masking a single large block. Replacing the xLSTM layer with a standard LSTM layer led to a small but consistent improvement, though this may be an artifact of using identical hyperparameters for both variants rather than a general property of the two layer types. This should be further investigated.

Finally, comparing checkpoints selected by best versus last test loss confirmed that early stopping on test loss is generally the right choice, but also revealed that quantile calibration keeps improving with additional training, suggesting that validation loss and quantile quality do not always improve together.

The strongly parameter-reduced custom TiRex model (1.6 million parameters) could not match the performance of the original 35-million-parameter pre-trained model on quantile predictions. Nevertheless, the results demonstrate that a much smaller model, trained with a suitable masking strategy and loss function, can still learn meaningful forecasting behavior on time series, which is encouraging for applications where computational resources are constrained. Also, fine-tuning the foundation model on a single setting or sequence is the right choice if more precise results are necessary.

Several topics remain open for future work. First, the hyperparameters used for the LSTM/sLSTM comparison and for the custom model in general were not individually tuned; a dedicated hyperparameter search per configuration could clarify whether the observed differences reflect genuine architectural properties or training artifacts. Second, the quantile collapse observed under autoregressive generation warrants further investigation, for example by combining autoregressive point-forecast generation with the original quantile head, or by exploring hybrid generation strategies. Third, scaling up the custom model and the amount of training data, currently limited by computational resources, could help establish where the performance gap to the full pre-trained model narrows. Finally, evaluating the fine-tuned and custom models on time series outside the training distribution would help assess how well the findings of this work generalize.

Final note: During the course of this work, an improved version of the TiRex foundation model, TiRex-2 (Podest et al. [10]), was released. TiRex-2 supports multivariate forecasting with past and future-known covariates, provides streaming capabilities, and achieves improved forecasting performance. Since it became available only after the experiments presented in this work had been completed, it was not included in the evaluation.

References

[1] Abdul Fatima Ansari, Caner Turkmen, Oleksandr Shchur, and Lorenzo Stella. 2024. Fast and Accurate Zero-Shot Forecasting with Chronos-Bolt and AutoGluon. AWS Machine Learning Blog. (December 2024). https://aws.amazon.com/blogs/machine-learning/fast-and-accurate-zero-shot-forecasting-with-chronos-bolt-and-autogluon/.

[2] J. Scott Armstrong. 1985. *Long-Range Forecasting: From Crystal Ball to Computer*. (2nd ed.). Wiley. ISBN: 978-0-471-82260-8.

[3] Andreas Auer, Patrick Podest, Daniel Klotz, Sebastian Böck, Günter Klambauer, and Sepp Hochreiter. 2025. TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning. (2025). arXiv: 2505.23719 [cs.LG]. https://arxiv.org/abs/2505.23719.

[4] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. 2024. xLSTM: Extended Long Short-Term Memory. (2024). arXiv: 2405.04517 [cs.LG]. https://arxiv.org/abs/2405.04517.

[5] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. (2024). arXiv: 2310.10688 [cs.CL]. https://arxiv.org/abs/2310.10688.

[6] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation*, 9, 8, 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[7] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. (2019). arXiv: 1711.05101 [cs.LG]. https://arxiv.org/abs/1711.05101.

[8] Marco Obermeier. 2026. FETS Dataset: Foundation Models Outperform Dataset-specific Machine Learning in Energy Time Series Forecasting. (April 2026). DOI: 10.5281/zenodo.19418721. https://doi.org/10.5281/zenodo.19418721.

[9] 2019. Open Power System data. (2019). DOI: 10.25832/time_series/2019-06-05. https://doi.org/10.25832/time_series/2019-06-05.

[10] Patrick Podest, Marco Pichler, Elias Bürger, Levente Zólyomi, Bernhard Voggenberger, Wilhelm Berghammer, Daniel Klotz, Sebastian Böck, Günter Klambauer, and Sepp Hochreiter. 2026. TiRex-2: Generalizing TiRex to Multivariate Data and Streaming. (2026). arXiv: 2607.01204 [cs.LG]. https://arxiv.org/abs/2607.01204.

[11] 2023. Renewable Energy and Electricity Demand Time Series Dataset with Exogenous Variables at 5-minute Interval. (2023). DOI: 10.17632/fdfftr3tc2.1. https://openenergyhub.ornl.gov/explore/dataset/renewable-energy-and-electricity-demand-time-series-dataset-with-exogenous-varia/information/.

Appendix A. An Appendix

A.1 Train and Validation Losses, Fine-tuning pre-trained model on single sequence

Train and Validation losses are visible in Table 5 and Figure 4.

Table 5: Train Losses - Pre-trained TiRex (single sequence)

Dataloader	Layer	Loss	Best Train Loss	Best Validation Loss
TiRex Original	slstm	quantile	200.12 ± 3.28	409.94 ± 4.34
TiRex Autoregressive	slstm	quantile	218.08 ± 3.68	404.46 ± 5.41
TiRex Original	slstm	mse	506,763 ± 28,471	2,048,799 ± 36,625
TiRex Autoregressive	slstm	mse	725,797 ± 38,896	2,029,340 ± 127,107
TiRex Original	slstm	l1	576.12 ± 18.21	1,031.99 ± 10.82
TiRex Autoregressive	slstm	l1	594.73 ± 9.14	1,005.34 ± 16.54

[a] Mean ± std across runs (lower loss is better).
[b] Each configuration was trained for 15 epochs in each of 5 independent runs.

A.2 Train and Validation Losses, Custom TiRex on big dataset

Train and Validation losses are visible in Table 6 and Figure 5. We see more stable results when training with CPM and an LSTM layer. It seems xLSTM has high variance due to exponential activations, and multiple runs are necessary to find the best model.

A.3 Train and Validation Losses, Custom TiRex on single sequence

Train and Validation losses are visible in Table 7 and Figure 6.

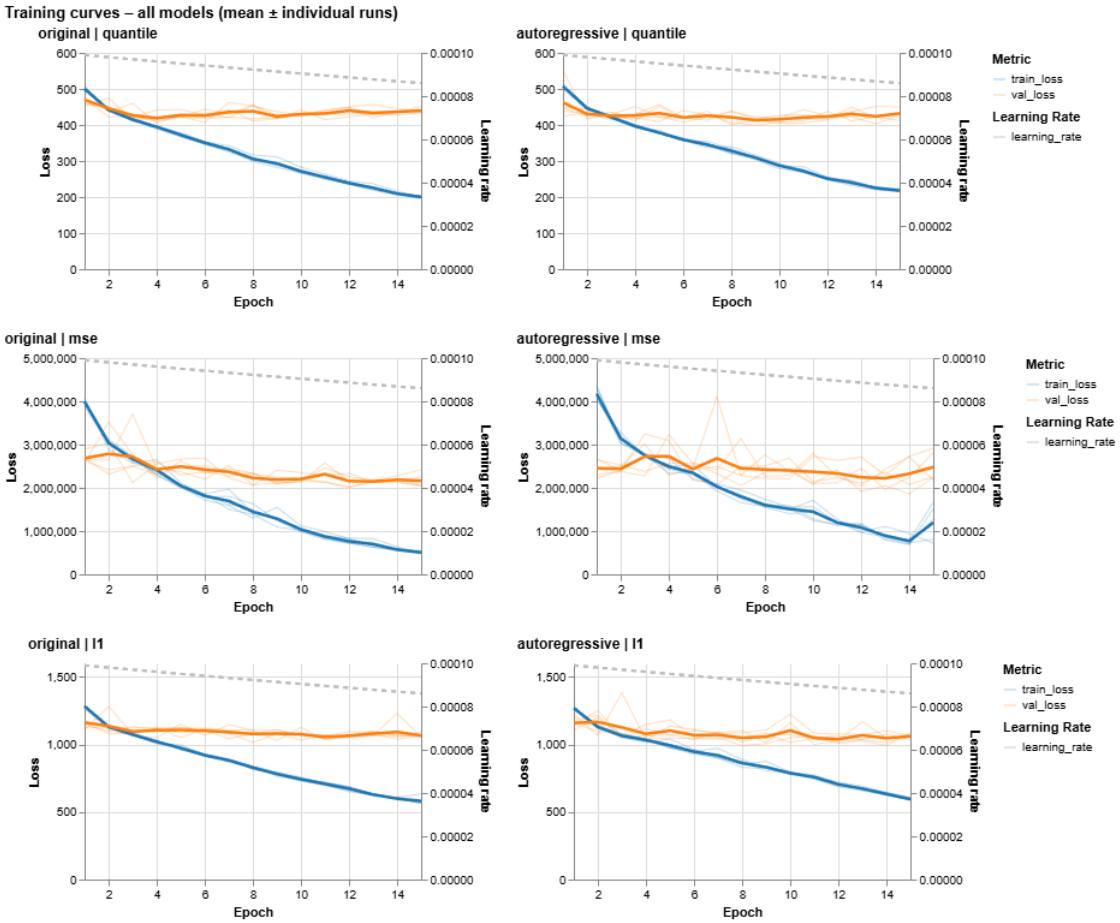


Figure 4: Train Losses - Pre-trained TiRex (single sequence)

Table 6: Train Losses - Custom TiRex (big dataset)

Dataloader	Layer	Loss	Best Train Loss	Best Validation Loss
MH	slstm	quantile	188.29 ± 109.07	384.48 ± 160.34
MH	lstm	quantile	52.07 ± 0.89	272.01 ± 3.83
cpm	slstm	quantile	99.30 ± 42.51	150.57 ± 3.91
cpm	lstm	quantile	40.64 ± 0.36	151.89 ± 2.35
MH	slstm	mse	1,562,264 ± 2,098,276	6,531,559 ± 3,209,992
MH	lstm	mse	129,727 ± 11,360	4,049,305 ± 193,178
cpm	slstm	mse	398,419 ± 341,756	2,087,947 ± 1,024,723
cpm	lstm	mse	53,482 ± 1,821	1,567,425 ± 43,922
MH	slstm	l1	518.86 ± 194.98	759.81 ± 206.19
MH	lstm	l1	155.81 ± 6.42	689.08 ± 18.21
cpm	slstm	l1	223.88 ± 83.65	453.50 ± 155.78
cpm	lstm	l1	111.46 ± 0.56	385.33 ± 4.22

[a] Mean ± std across runs (lower loss is better).
[b] Each configuration was trained for 100 epochs in each of 5 independent runs.

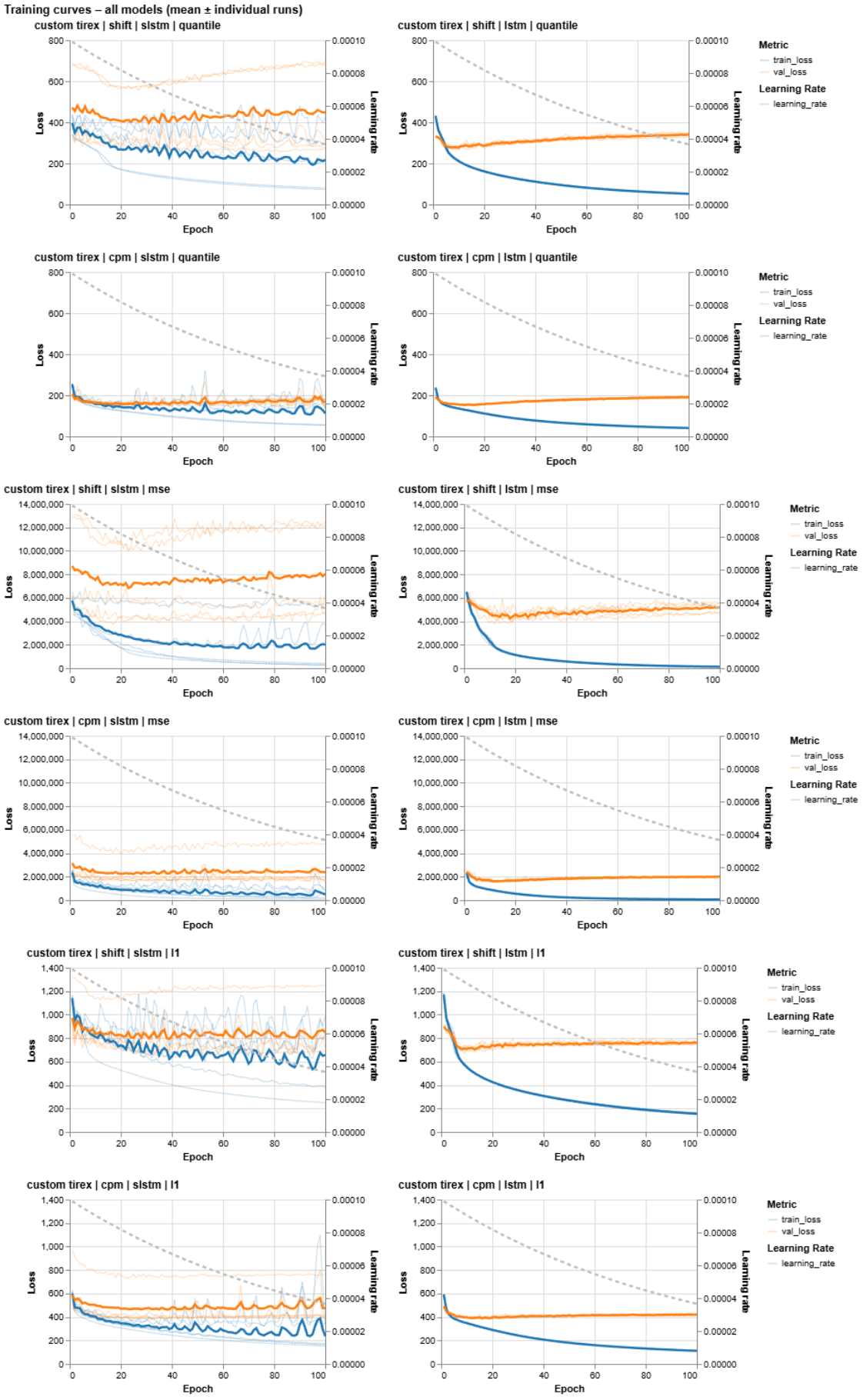


Figure 5: Train Losses - Custom TiRex (big dataset)

Table 7: Train Losses - Custom TiRex (single sequence)

Dataloader	Layer	Loss	Best Train Loss	Best Validation Loss
MH	slstm	quantile	543.71 ± 7.83	612.24 ± 6.96
MH	lstm	quantile	399.73 ± 3.77	545.44 ± 4.27
MH	slstm	mse	3,483,126.45 ± 72,156.17	4,867,615 ± 16,038
MH	lstm	mse	1,219,947 ± 69,377.99	3,730,064± 200,138
MH	slstm	l1	1,326.44 ± 17.14	1,551.57 ± 20.72
MH	lstm	l1	1,009.09 ± 31.85	1,460.09 ± 39.63

[a] Mean ± std across runs (lower loss is better).
[b] Each configuration was trained for 100 epochs in each of 5 independent runs.

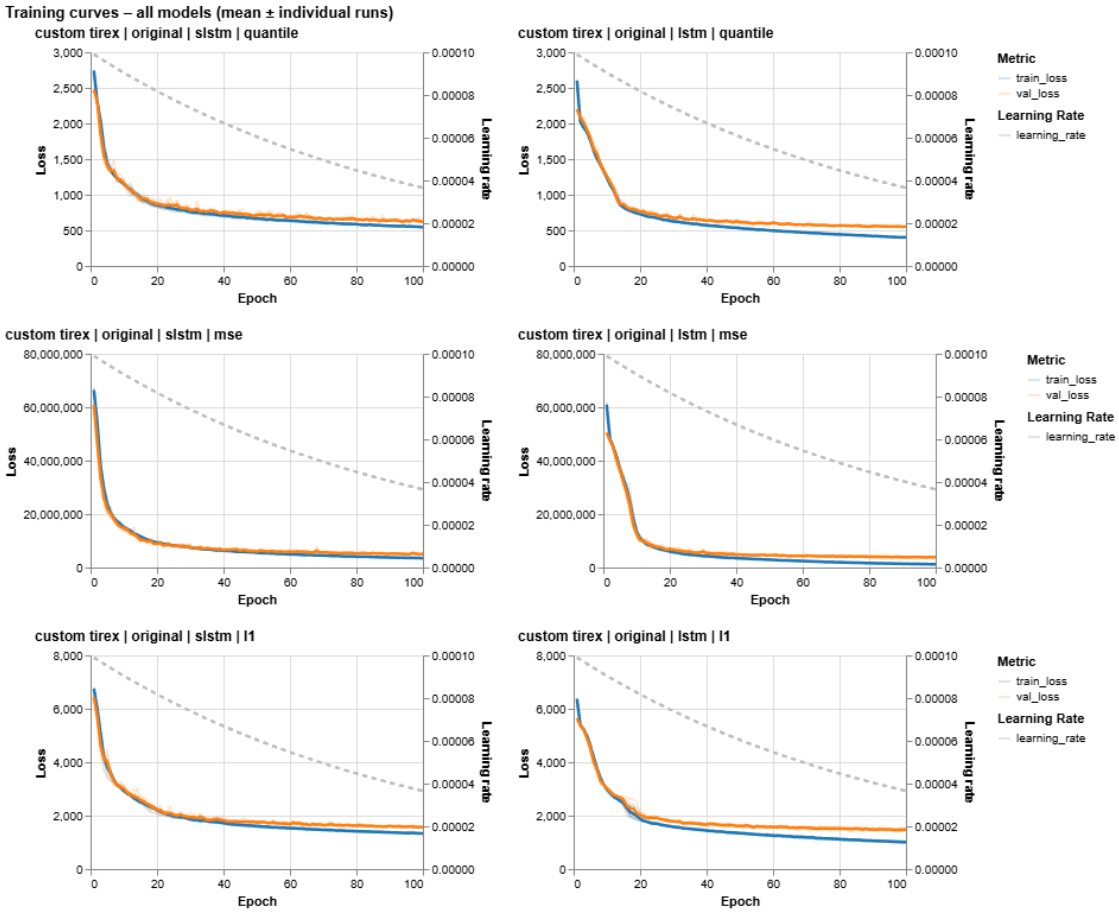


Figure 6: Train Losses - Custom TiRex (single sequence)